# Operating Systems
# Project Phase 3 Report

Farah Elnakhal - fqe9080
Myra Rafiq - mr7316

17th April 2026

# 1 Architecture and Design

In Phase 3, we extended our Phase 2 client-server system to support multiple clients at the same time using multithreading. The overall design follows a client-server model where clients send shell commands to the server, and the server executes them and returns the output.
The main design decision was to use a thread per client model. Each time a client connects, the server creates a new thread using `pthread_create()`. This allows multiple clients to interact with the server concurrently instead of waiting for one another.
Each thread is responsible for:

- Receiving commands from its client
- Executing the command using previous phases functions (`execute_simple_command()` or `execute_pipeline()`)
- Sending the output back to the client

To manage client-specific data, we introduced a small structure that stores the client socket, client ID, and address information (IP and port). This ensures that each thread works independently.
We also used a global counter with a mutex lock to assign unique client IDs safely. This avoids race conditions when multiple clients connect at the same time.

# 2 Implementation Highlights

The main improvement in this phase was adding multithreading to the server so it can handle multiple clients at once.
Instead of handling clients one by one, the server now creates a new thread for each client using `pthread_create()`. Each thread runs the `handle_client()` function and processes commands independently.
We passed client information to each thread using dynamically allocated memory. This avoids issues where multiple threads might access the same variable.
To ensure each client gets a unique ID, we used a mutex lock when updating the shared counter. This prevents conflicts when multiple clients connect simultaneously.
We also updated the logging format to include the client number, IP address, and port, as required. This makes it easier to track activity across multiple clients.
Basic error handling was added for:

- Thread creation failures
- Socket read/write errors
- Client disconnections

The `exit` command was handled explicitly. When received, the server closes the connection and terminates the thread cleanly.
Finally, threads were detached using `pthread_detach()` so that resources are automatically freed after execution.

# 3 Execution Instructions

## 3.1 Prerequisites

The following tools must be installed on the remote Linux server:

- `gcc` – GNU C compiler
- `make` – build automation
- `netcat` (`nc`) – used by `test.sh`
- Standard Unix utilities: `ls`, `cat`, `grep`, `sort`, `wc`, `printf`

## 3.2 Compilation

Navigate to the project directory and run:

```
make clean
make
```

This produces three executables: `myshell` (local Phase 1 shell), `server` (multithreaded Phase 3 server), and `client`. The `server` binary is linked with `-lpthread`.

## 3.3 Running the server

```
./server
```

The server listens on port 8080 by default and prints:

```
[INFO] Server started, waiting for client connections...
```

It accepts an unlimited number of simultaneous clients. Press `Ctrl+C` to stop.

## 3.4 Running the client

In one or more separate terminals while the server is running:

```
./client
```

The client connects to `127.0.0.1:8080`, displays a `$` prompt, and accepts commands exactly as in Phases 1 and 2. Type `exit` to disconnect.

## 3.5 Running the test suite

```
chmod +x test.sh
./test.sh
```

The script compiles the project, starts the server on an automatically chosen free port (9000–9999 to avoid conflicts on shared servers), runs all test cases, and prints a final summary.

# 4 Testing

## 4.1 How the test script works

`test.sh` is fully automated. Its main steps are:

1. Compile with `make`.
2. Find a free port in the range 9000–9999 using `ss` and `lsof`.
3. Patch the port into a temporary copy of `server.c` with `sed`, recompile, and start the server in the background.
4. Run all test cases, each using `nc` to send commands and read responses.
5. Kill the server and clean up temporary files.
6. Print a final pass/fail summary.

Each test calls one of two helper functions:

- `check`: passes if the expected string appears anywhere in the output.
- `check_absent`: passes if a given string does *not* appear in the output.

ANSI escape codes are stripped from the server log with `sed` before string matching so colour codes do not interfere with `grep`.

## 4.2 Test categories

**Phase 1 regressions.** Eight tests confirm basic shell functionality still works end-to-end through the network: `echo`, `pwd`, `ls`, `ls -l`, `wc -l`, single pipe, pipe to `grep`, and pipe with `sort`.
**Simultaneous clients.** Three client processes are launched as background sub-shells at the same time. Client 1 runs `ls -l`, Client 2 runs the invalid command `unknowncmd`, and Client 3 runs `pwd`. All three are waited on before checking output, verifying that the server handles concurrent connections correctly.
**Server log format.** The server log is checked for eleven required properties, including all five tag types, the `[Client #N -- IP:PORT]` label, the IP address, the `Assigned to Thread-` string, and the correct error message format.
**Edge cases.** Four additional tests check that the server survives empty input, that `ls` on a missing file returns the expected error, and that multi-stage pipelines work correctly over the network.

## 4.3 Test results

All test cases pass. This can be seen in the screenshot below:

```
========================================
       PHASE 3 TESTING
========================================

Phase 3 using port: 9802
[PASS] Phase 3: server starts with multithreading
[PASS] Phase 3: Client 1 (ls -l) works concurrently
[PASS] Phase 3: Client 2 gets descriptive error (Command not found: unknowncmd)
[PASS] Phase 3: Client 3 (pwd) works concurrently
[PASS] Phase 3: Server assigns Thread IDs in [INFO] line
[PASS] Phase 3: Server labels clients with Client #N
[PASS] Phase 3: Server logs client IP address
[PASS] Phase 3: [RECEIVED] tag present in server log
[PASS] Phase 3: [EXECUTING] tag present in server log
[PASS] Phase 3: [OUTPUT] tag present in server log
[PASS] Phase 3: [ERROR] tag present for unknown command
[PASS] Phase 3: [ERROR] line names the bad command
[PASS] Phase 3: Multiple clients handled simultaneously (>= 3 IDs seen)
[PASS] Phase 3: Three-stage pipeline works remotely
[PASS] Phase 3: Remote error message for bad path
[PASS] Phase 3: Server still running after concurrent load
```

Figure 1: Phase 3 Test Results - All Tests Passing

# 5 Challenges

**Missing [ERROR] tag and generic error messages.** The most visible bug from Phase 2 was that the server produced no `[ERROR]` line and the client received only `"Command not found."` with no command name. The root cause is that `simple.c` calls `fprintf(stderr, "Command not found.\n")` after `execvp()` fails, and that string flows through `execute_and_capture()` unchanged. The fix was to detect the phrase in `server.c`, extract the command name, build a descriptive message, and log `[ERROR]` before sending the enriched string to the client. This keeps all formatting logic in `server.c` without modifying the shared Phase 1 files.

**Port conflicts on shared servers.** Port 8080 is frequently occupied on shared SSH servers. The test script probes ports 9000–9999 and dynamically patches the server source with `sed` before compiling the test binary, making the suite portable without manual intervention.

**Race condition on client IDs.** When two clients connect within milliseconds of each other, both threads can read `client_counter` before either increments it, producing duplicate IDs. Wrapping the increment in a mutex eliminates the race.

**Thread stack variable aliasing.** An early implementation passed a pointer to a local `client_info_t` on the main thread's stack to `handle_client()`. By the time the new thread read the struct, the main thread had already overwritten it with the next client's data. The fix was to `malloc()` a fresh struct for every client and have the thread `free()` it after copying the fields it needs.

# 6 Division of Tasks

Myra was responsible for implementing multithreading in the server including creating the thread-per-client model, handling client communication inside threads, managing thread-safe client IDs, and adding basic error handling.

Farah was responsible for fixing the errors from phase 2, implementing the required server logging format, including all message types such as [INFO], [RECEIVED], [EXECUTING], [OUTPUT], and [ERROR], along with client IP, port, and thread details. She also handled the testing script.
Both partners collaborated on integrating the final system, verifying that all features worked correctly by testing, and completing the report.